Parcial Patrones de software

Johan Sebastian Jaimes Burbano

Alejandra Mayorga Barboza

Diseño de Patrones de software

Eliecer Montero Ojeda

FACULTAD CIENCIAS E INGENIERIAS UNIDADES

TECNOLOGICAS DE SANTANDER

BUCARAMANGA – SANTANDER

2026

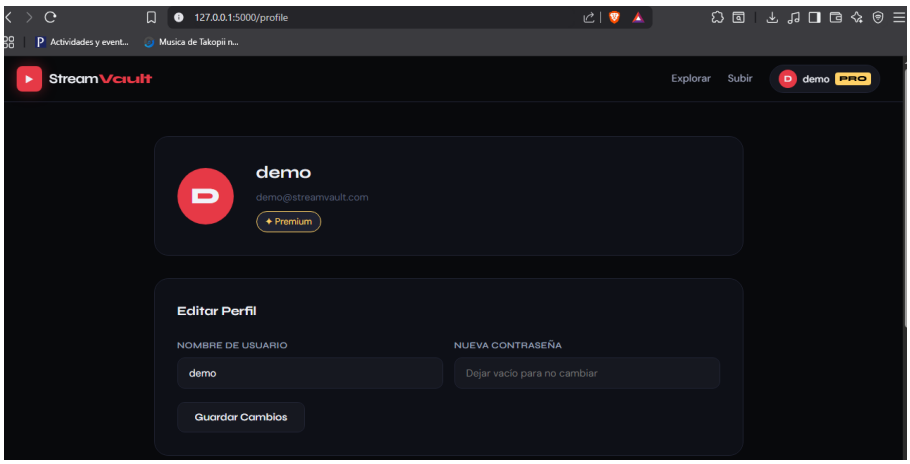
Abstract Factory

```python
from patterns import DatabaseConfig, UserFactory, VideoFactory, get_subscription_factory
from flask import Flask, render_template, request, jsonify, redirect, url_for, session, Response, abort
from flask_sqlalchemy import SQLAlchemy
from flask_bcrypt import Bcrypt
from functools import wraps
import os, uuid, json
from datetime import datetime
from werkzeug.utils import secure_filename
```

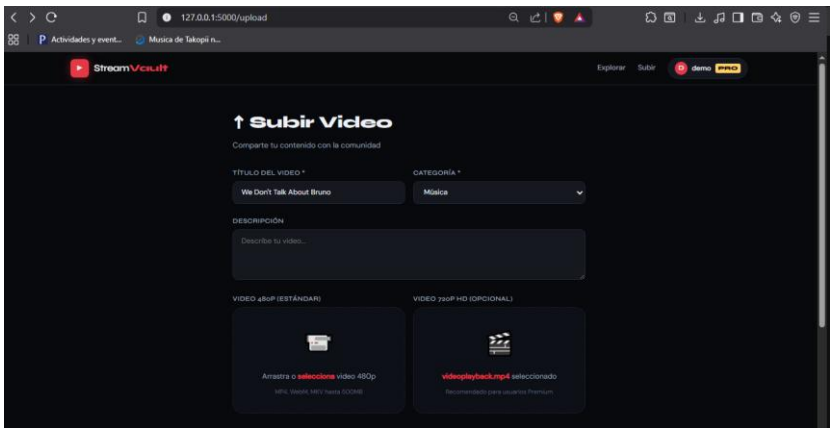
Solo se agregó get_subscription_factory a la línea de import que ya existía. Esta función es la que actúa como punto de entrada al Abstract Factory — recibe el plan del usuario como texto ('free' o 'premium') y devuelve el factory concreto correspondiente. Sin agregarla al import, Python no sabría dónde buscarla cuando la ruta /watch la necesite.

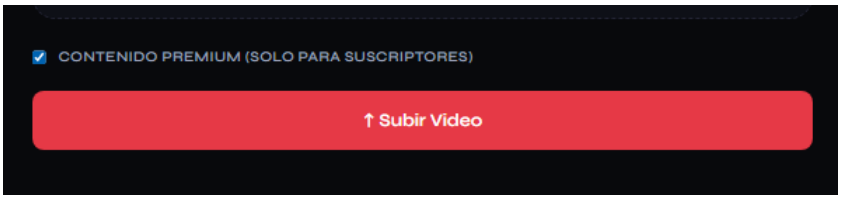
```python
@app.route('/watch/<int:video_id>')
@login_required
def watch(video_id):
    user = get_current_user()
    video = Video.query.get_or_404(video_id)

    # — Abstract Factory ————————————————————————————————————————
    factory = get_subscription_factory(user.subscription)
    player  = factory.create_player()
    access  = factory.create_access()
    # ——————————————————————————————————————————————————————————
```

Ejemplo de usuario Premium:

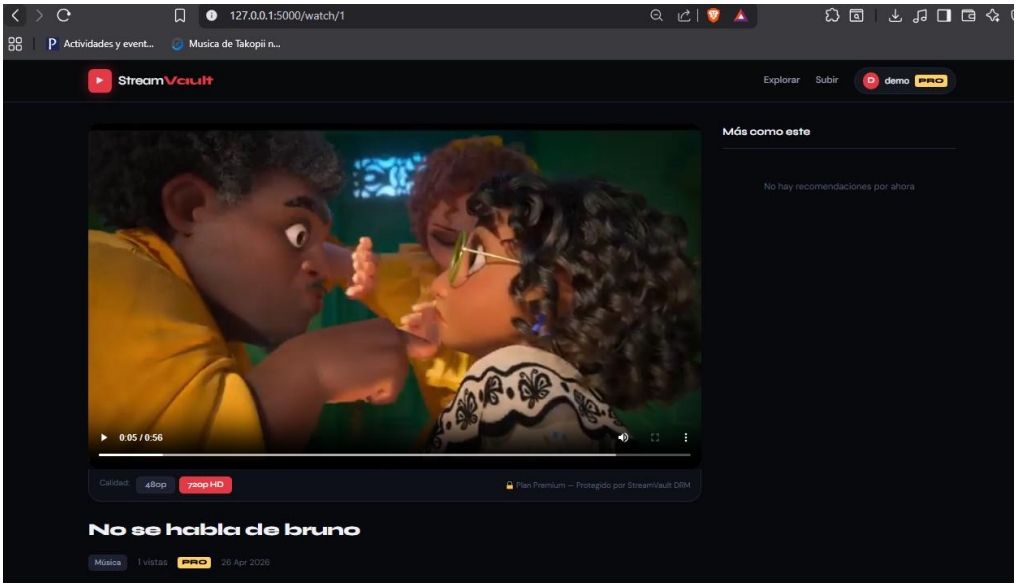


Apartado para subir los videos siendo premium para subirlos en diferentes calidades y marca si lo puede ver tanto usuarios free como premium

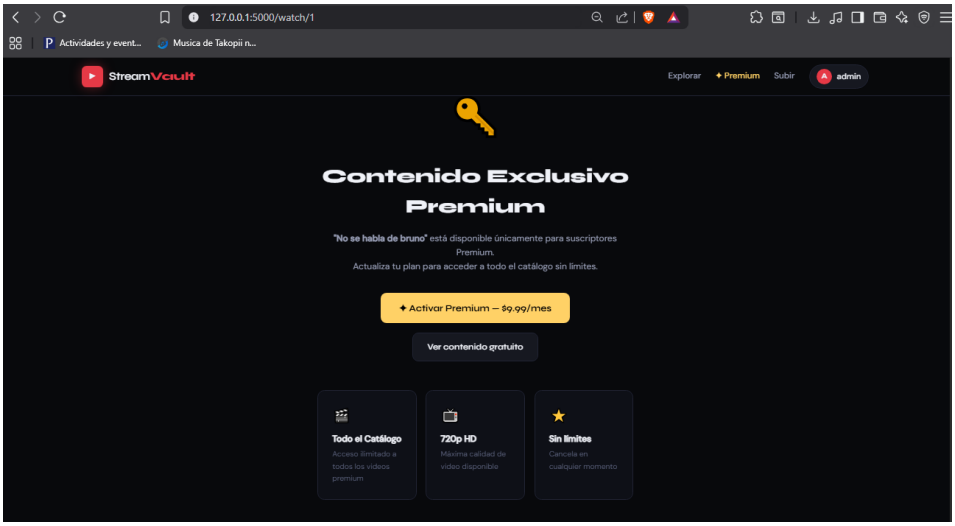




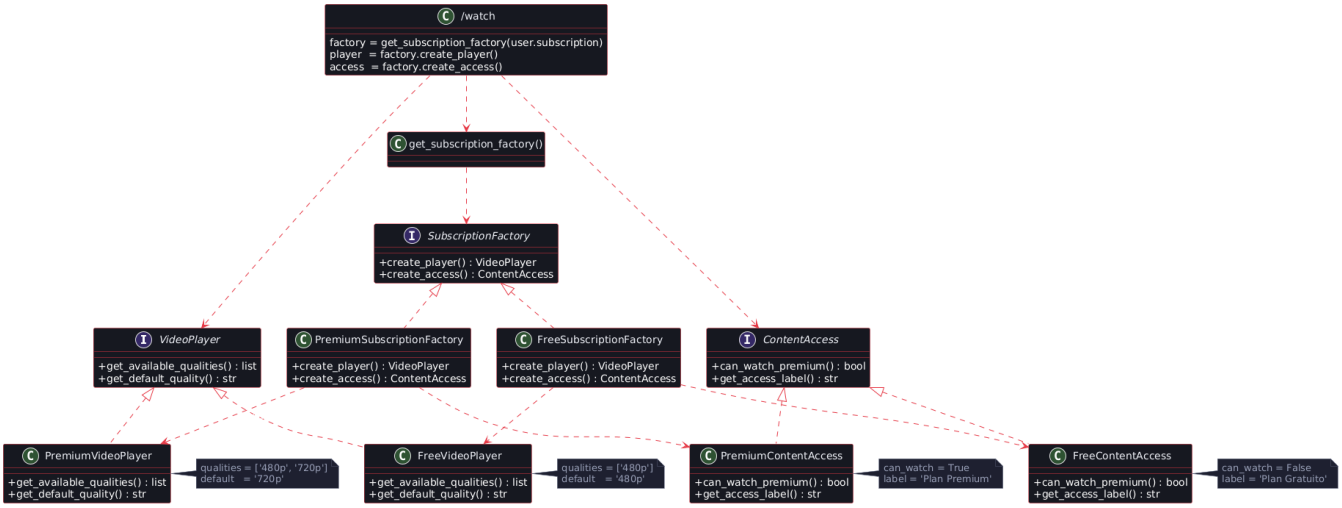
- Se muestran los dos botones 480p y 720p HD
- La etiqueta dirá Plan Premium



La vista de un usuario Free al intentar ver el mismo video:



UML



Builder

rutas API

ahora se encadenan **set_success(True)** y **set_redirect('/')** de forma explícita, dejando claro que una respuesta exitosa de registro siempre lleva esos dos campos. Los errores de validación también pasaron por el Builder usando **set_error(),** que automáticamente marca success como False sin necesidad de escribirlo manualmente.

```python
if not result.success:
    return jsonify(APIResponseBuilder().set_error(result.error).build()), 400

response = (APIResponseBuilder()
            .set_success(True)
            .set_redirect('/')
            .build())
return jsonify(response)  # ← BUILDER
```

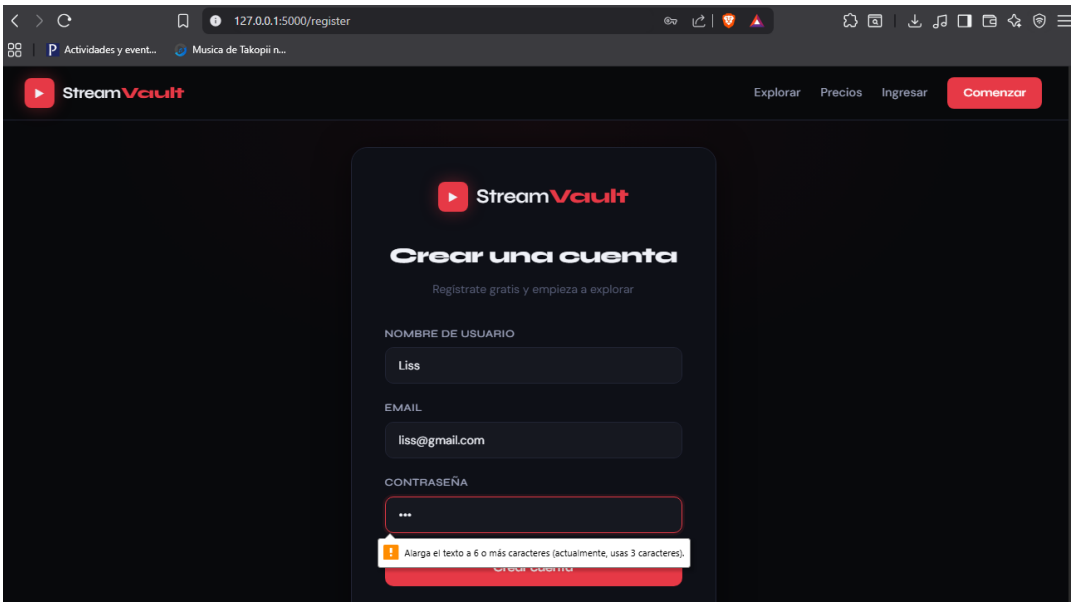
Cuando se intenta registrar un usuario con datos inválidos:

```
13 requests | 8.2 kB transferred | 246 kB resources | Finish: 6.5 min | DOMContentLoaded: 64 ms
Console  What's new
top ▾  Filter  Default levels ▾  3 Issues: 3
> apiFetch('/api/register', 'POST', {username:'test', email:'test@test.com',
  password:'123'}).then(console.log).catch(console.error)
<· ▸ Promise {<pending>}
✕ ▸ POST http://127.0.0.1:5000/api/register 400 (BAD REQUEST)  main.js:24
  ▸ {error: 'La contraseña debe tener al menos 6 caracteres', success: false}
>
```

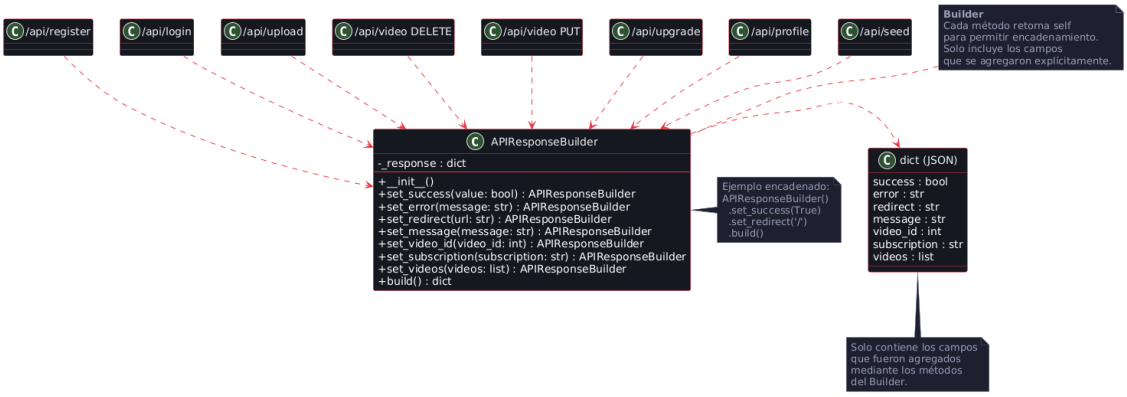
Cuando se cumple se hace con datos validos:

```
> apiFetch('/api/register', 'POST', {username:'usuarioprueba',
  email:'prueba@test.com',
  password:'password123'}).then(console.log).catch(console.error)
<· ▸ Promise {<pending>}
  ▸ {redirect: '/', success: true}
>
```

Pantalla de registro con datos inválidos:



UML



Decorator

Cada filtro es una capa que se envuelve sobre la anterior.

- BaseVideoQuery devuelve todos los videos.
- CategoryFilterDecorator toma esos videos y filtra los que no son de la categoría pedida.
- SearchFilterDecorator toma esos resultados y filtra los que no tienen el texto buscado.

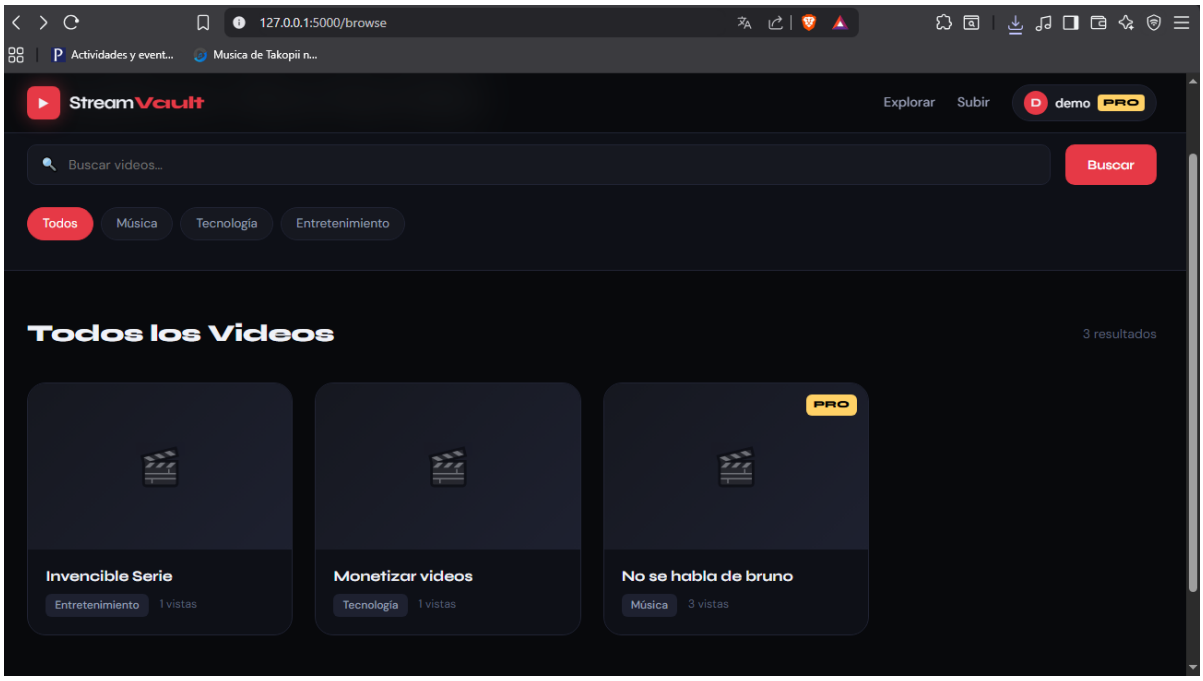
- PremiumFilterDecorator toma esos resultados y oculta el contenido premium si el usuario es free. Cada capa recibe lo que dejó la anterior y agrega su propia condición encima.

Lo que cambia visualmente

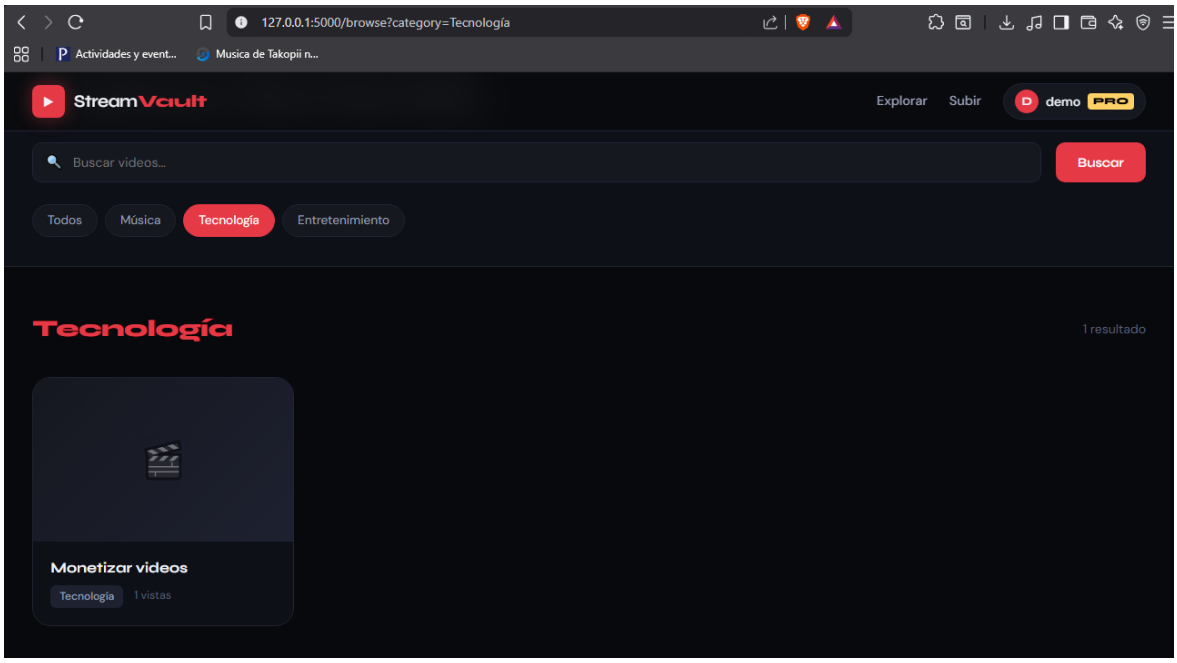
Antes la consulta era una cadena de SQLAlchemy que se iba modificando. Ahora es un objeto que se va envolviendo. La diferencia clave es que agregar un filtro nuevo no modifica ningún código existente — solo se crea una nueva clase VideoQueryDecorator y se agrega su if en la ruta. Los filtros anteriores no se tocan.

```python
@app.route('/browse')
def browse():
    user     = get_current_user()
    category = request.args.get('category', '')
    search   = request.args.get('search', '')
    popular  = request.args.get('popular', '')

    # — Decorator ————————————————————————————————————————————————
    # Se inyecta Video para evitar problemas de contexto con Flask-SQLAlchemy.
    # Se parte de la consulta base y se encadenan decoradores según
    # los filtros activos. Cada decorador agrega su condición sin
    # modificar los anteriores.
    query = BaseVideoQuery(Video)

    if category:
        query = CategoryFilterDecorator(query, category)
    if search:
        query = SearchFilterDecorator(query, search)
    if user:
        query = PremiumFilterDecorator(query, user.subscription)
    if popular:
        query = PopularFilterDecorator(query)

    videos = query.execute()
```

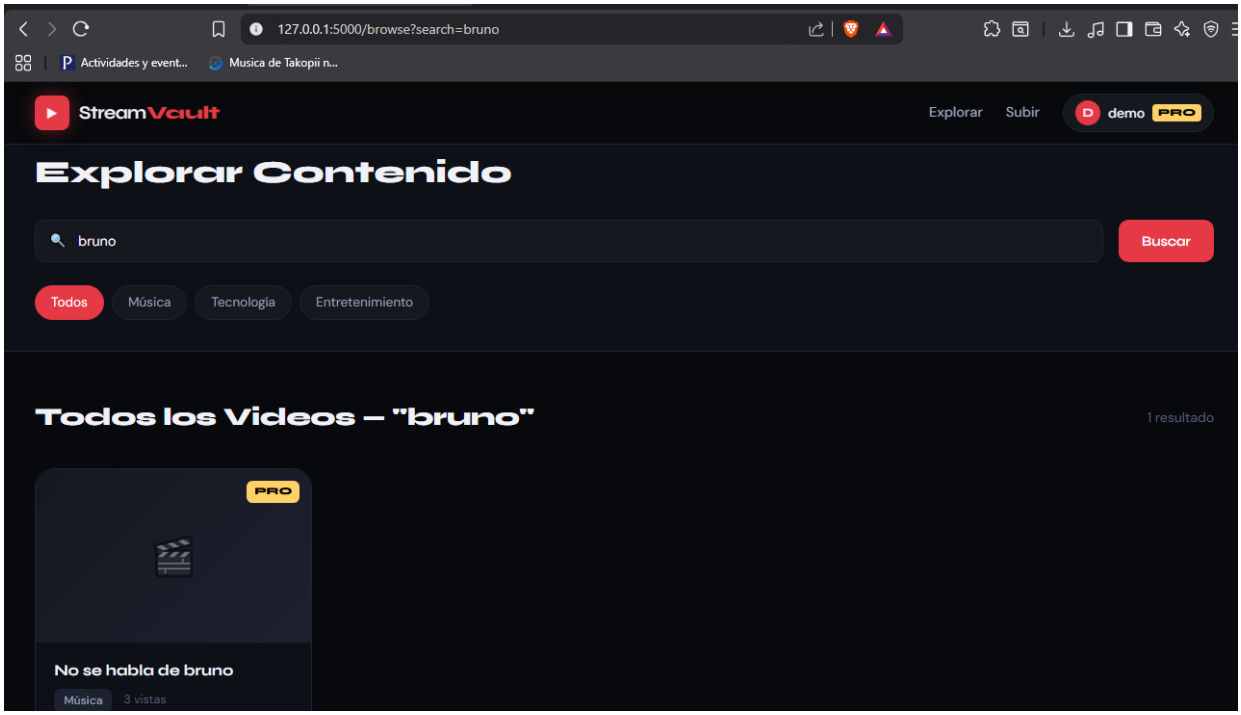
Se muestran todos lo videos de todas las categorias



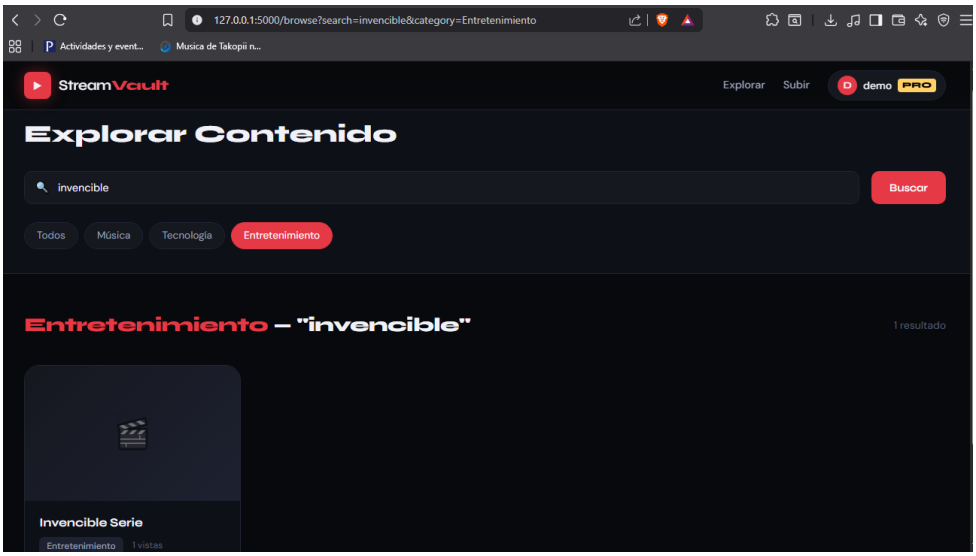
Filtro por categoría la URL cambia de http://127.0.0.1:5000/browse a http://127.0.0.1:5000/browse?category=Tecnología



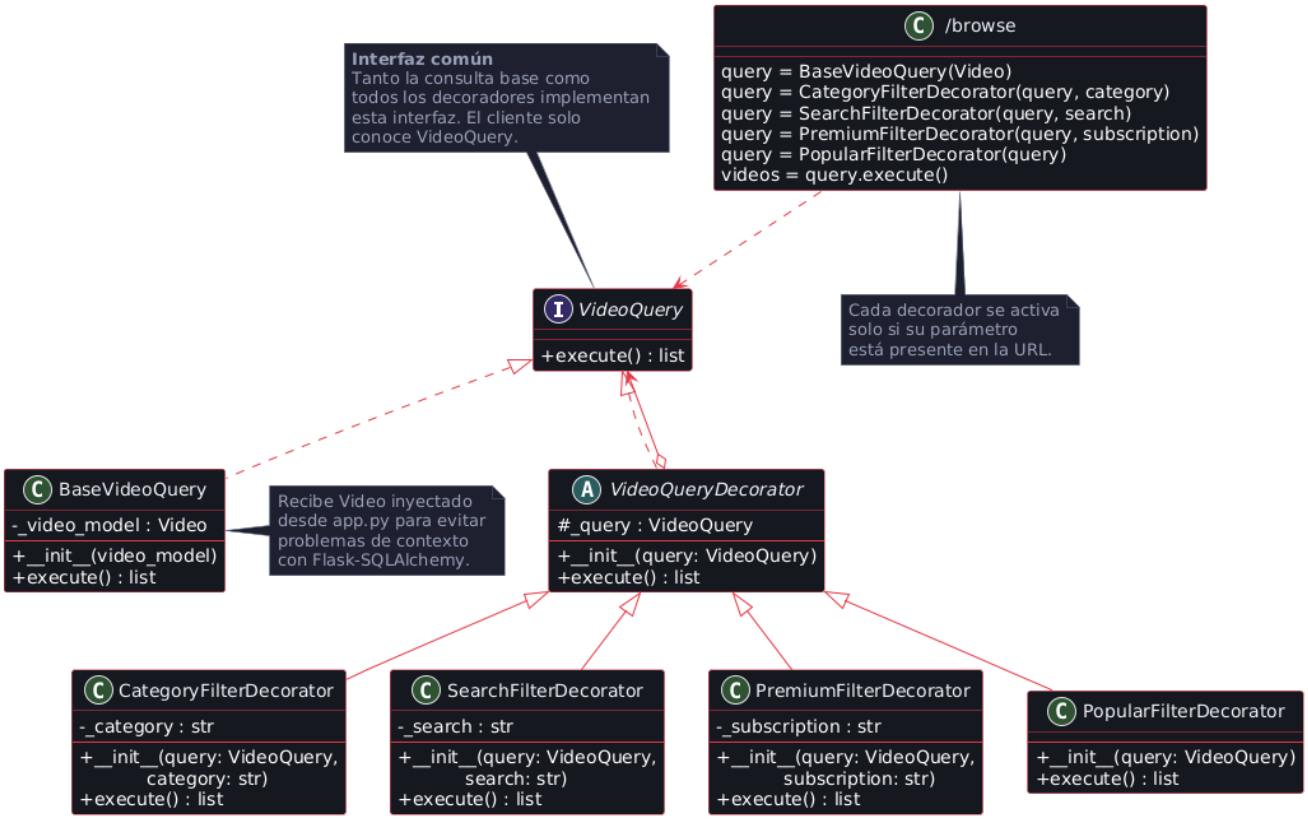
Solo aparecerán videos que contengan ese texto:



Y para Búsquedas combinadas con filtros:



UML



Singleton

Antes la URI estaba escrita directamente como texto 'sqlite:///streamvault.db'. El problema es que si en 10 lugares del código necesitas esa cadena, la repites 10 veces y si cambias de base de datos tienes que buscarla en todos lados.

```
app.config['SQLALCHEMY_DATABASE_URI'] =
'sqlite:///streamvault.db'
```

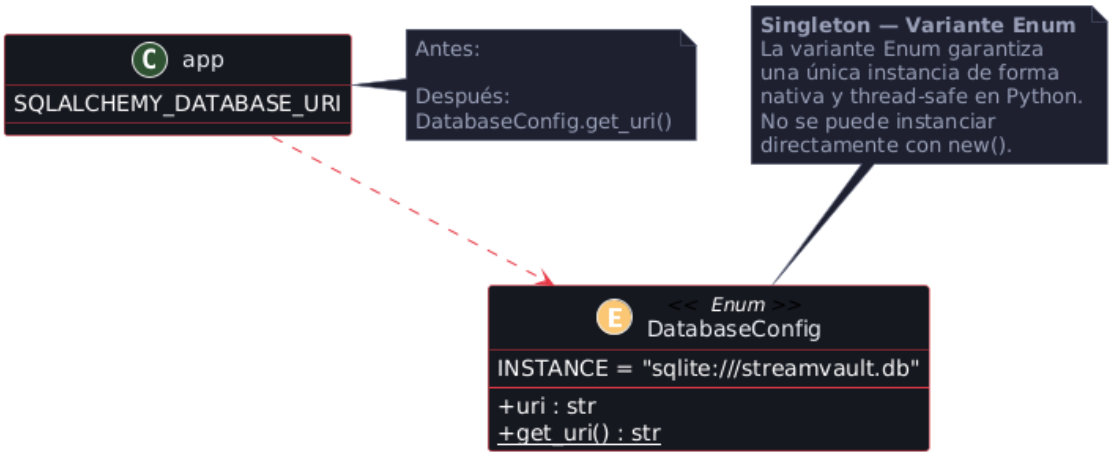
Ahora hay una única fuente de verdad. DatabaseConfig es un Enum, y en Python los Enum son Singleton por naturaleza — solo existe una instancia de INSTANCE en toda la ejecución del programa. Si mañana cambias a PostgreSQL, solo tocas patterns.py en un solo lugar.

```python
app = Flask(__name__)
app.config['SECRET_KEY'] = 'streamvault-secret-key-2024'
app.config['SQLALCHEMY_DATABASE_URI'] = DatabaseConfig.get_uri()  # ← SINGLETON
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['UPLOAD_FOLDER'] = 'static/uploads'
app.config['MAX_CONTENT_LENGTH'] = 500 * 1024 * 1024
ALLOWED_EXTENSIONS = {'mp4', 'webm', 'mkv', 'avi', 'mov'}
```

```python
class DatabaseConfig(Enum):
    INSTANCE = "sqlite:///streamvault.db"

    @property
    def uri(self):
        return self.value

    @staticmethod
    def get_uri():
        return DatabaseConfig.INSTANCE.uri
```

Por eso se queda en el proyecto por la comodidad a la hora del manejo de la base de datos.

UML



Factory Method

Para Usuario

Antes app.py hacía dos cosas: crear el objeto User Y encriptarle la contraseña. Eso es demasiada responsabilidad para una ruta HTTP.

```
user = User(username=username, email=email, subscription='free')
user.set_password(password)
```

El Factory recibe los parámetros, crea el objeto User, llama set_password() para encriptar la contraseña, y devuelve el objeto listo. La ruta no sabe que existe set_password() — ese detalle quedó encapsulado dentro del Factory.

```
user = UserFactory.create(username=username,
                          email=email,
                          password=password)
```

```python
class UserFactory:
    @staticmethod
    def create(username: str, email: str, password: str,
               subscription: str = 'free'):
        from app import User
        user = User(username=username, email=email, subscription=subscription)
        user.set_password(password)
        return user
```

Para Video

```
video = Video(title=title, category=category,
uploader_id=user.id,description=description,
is_premium=is_premium)
```

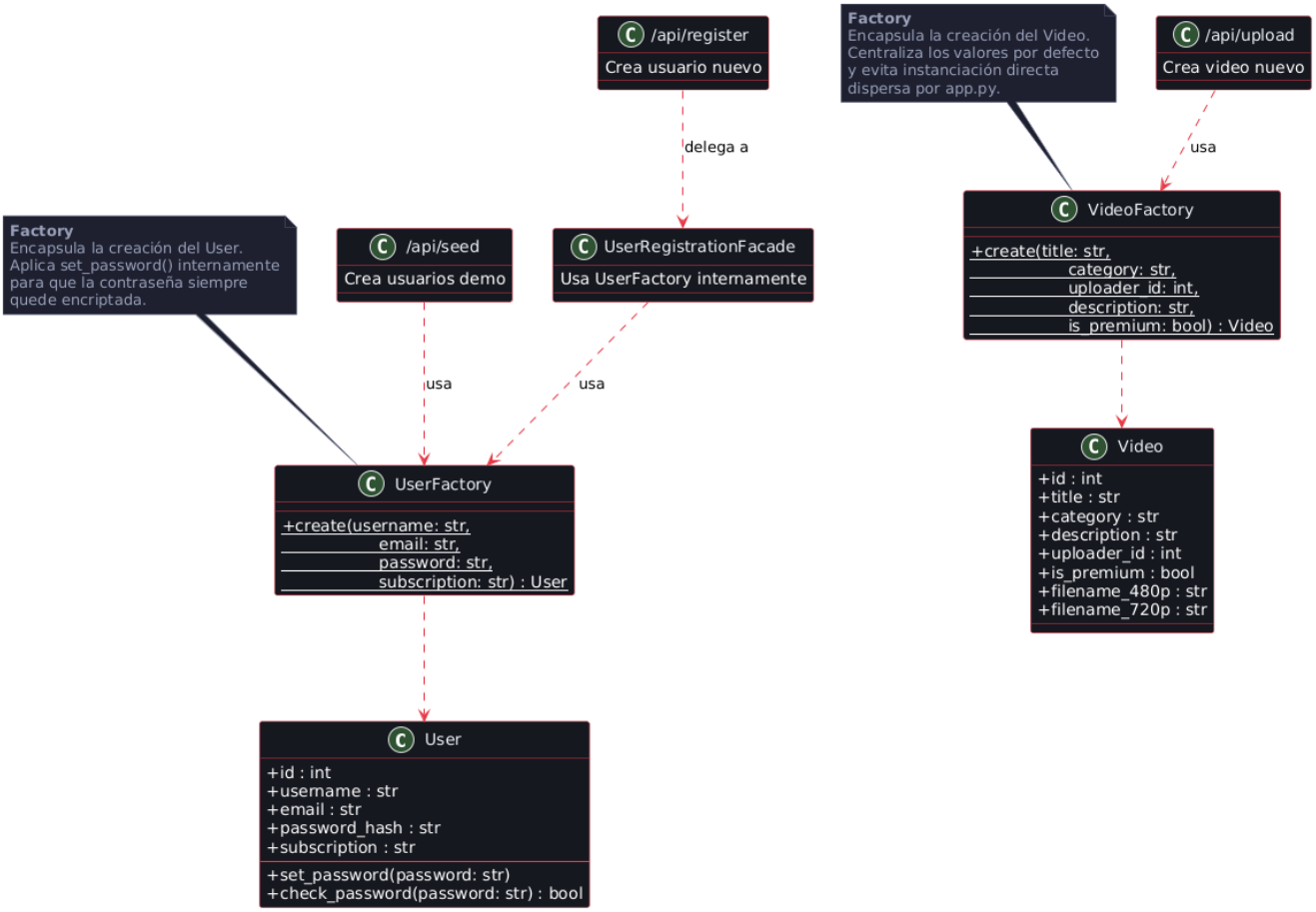
Antes app.py construía el objeto Video con todos sus parámetros directamente. Si el modelo Video cambia tenías que buscarlo en app.py y modificarlo ahí.

Ahora ese conocimiento vive en VideoFactory. app.py solo pasa los datos y recibe el objeto listo.

```python
@app.route('/api/upload', methods=['POST'])
@login_required
def upload_video():
    user = get_current_user()
    title = request.form.get('title', '').strip()
    description = request.form.get('description', '').strip()
    category = request.form.get('category', '').strip()
    is_premium = request.form.get('is_premium') == 'true'

    if not title or not category:
        return jsonify(APIResponseBuilder().set_error('Título y categoría

    video = VideoFactory.create(  # ← FACTORY
        title=title,
        category=category,
        uploader_id=user.id,
        description=description,
        is_premium=is_premium
    )
```

Sin el Factory, la lógica de encriptación estaba dispersa. Si alguien agregaba una ruta nueva que creara usuarios y olvidaba llamar set_password(), las contraseñas quedaban guardadas en texto plano sin ninguna advertencia. Con el Factory ese error es imposible — la encriptación siempre ocurre dentro de create().

UML



Facade

Antes la ruta mezclaba validación, lógica de negocio, acceso a la base de datos y manejo de sesión en un mismo lugar, lo que la hacía larga y difícil de leer:

```
# 4 validaciones manuales
if not username or not email or not password:
    return jsonify({'error': '...'}), 400
if len(password) < 6:
    return jsonify({'error': '...'}), 400
if User.query.filter_by(username=username).first():
    return jsonify({'error': '...'}), 400
```

```
if User.query.filter_by(email=email).first():
    return jsonify({'error': '...'}), 400


# Factory manual
user = UserFactory.create(username=username, email=email,
password=password)
# BD manual
db.session.add(user)
db.session.commit()
# Sesión manual
session['user_id'] = user.id
```

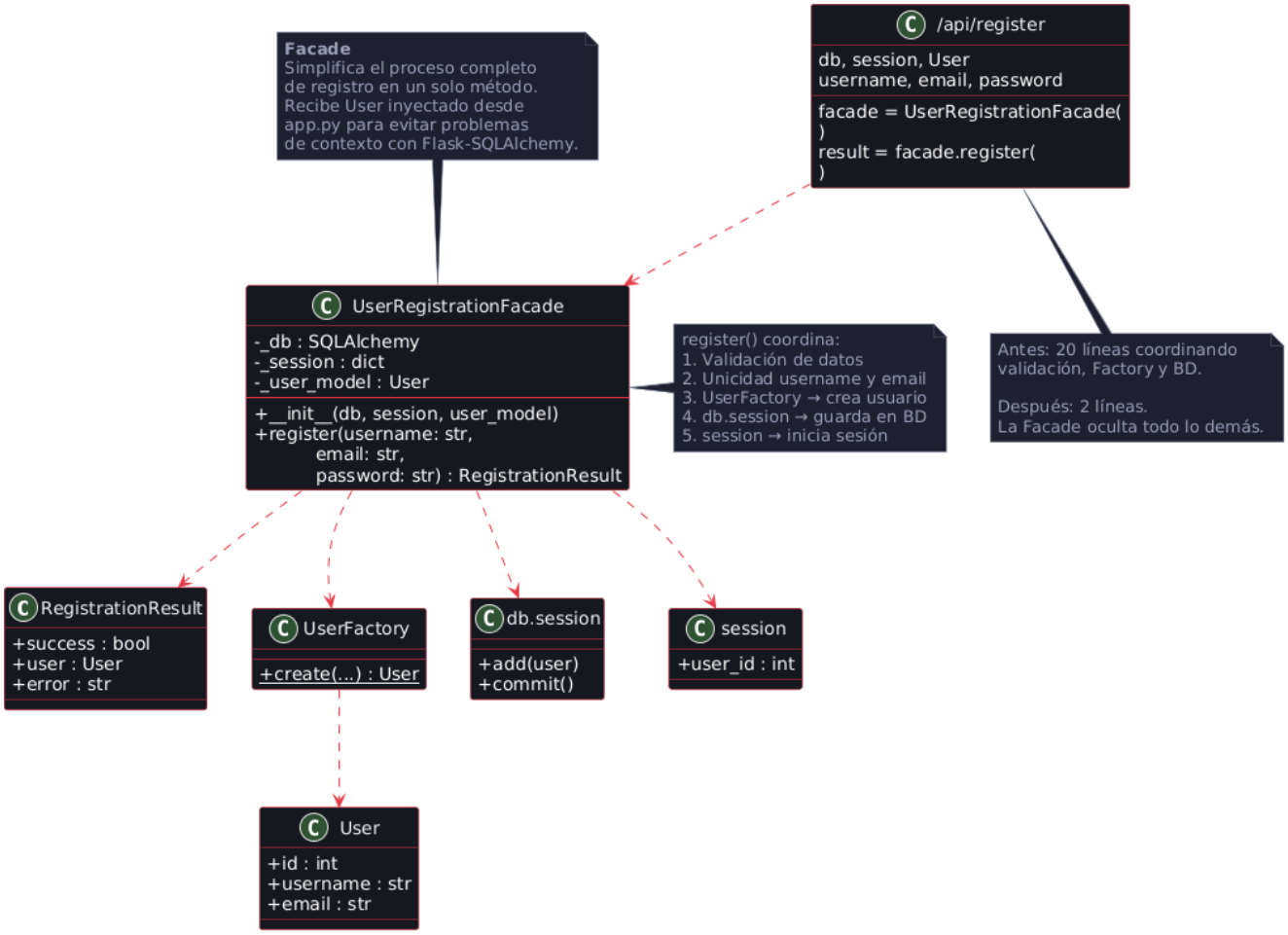
Después con la Facade:

UserRegistrationFacade tiene un método register() que ejecuta los 5 pasos en orden. El paso 1 valida que los campos no estén vacíos. El paso 2 verifica que el username y el email no existan en la base de datos usando el modelo User inyectado. El paso 3 llama a UserFactory.create() para crear el objeto usuario. El paso 4 guarda en la base de datos y hace commit. El paso 5 guarda el user_id en la sesión de Flask

```
facade = UserRegistrationFacade(db, session, User)
result = facade.register(username=username, email=email, password=password)
# ──────────────────────────────────────────────────────────────────────────

if not result.success:
    return jsonify(APIResponseBuilder().set_error(result.error).build()), 400
```

Con la Facade toda esa complejidad queda encapsulada en un solo objeto. Si en el futuro el proceso de registro necesita un paso nuevo como verificación de email o asignación de avatar por defecto, solo se agrega ese paso dentro de register() y la ruta no se toca.

UML


Facade
Simplifica el proceso completo
de registro en un solo método.
Recibe User inyectado desde
app.py para evitar problemas
de contexto con Flask-SQLAlchemy.
/api/register
db, session, User
username, email, password
facade = UserRegistrationFacade(
)
result = facade.register(
)
UserRegistrationFacade
-_db : SQLAlchemy
-_session : dict
-_user_model : User
+__init__(db, session, user_model)
+register(username: str,
email: str,
password: str) : RegistrationResult
register() coordina:
1. Validación de datos
2. Unicidad username y email
3. UserFactory → crea usuario
4. db.session → guarda en BD
5. session → inicia sesión
Antes: 20 líneas coordinando
validación, Factory y BD.
Después: 2 líneas.
La Facade oculta todo lo demás.
RegistrationResult
+success : bool
+user : User
+error : str
UserFactory
+create(...) : User
db.session
+add(user)
+commit()
session
+user_id : int
User
+id : int
+username : str
+email : str